SUPPORT VECTOR MACHINES (SVMs)

1.0 INTRODUCTION

A Support Vector Machines (Support Vector Network) is a robust, flexible, and powerful Machine Learning Algorithm, which has vast applications in classification (Both linear and nonlinear) and regression ML problems, and also in outliers' detection. Properly introduced in 1963 by Vapnik, these features make it one of the most popular models in Machine Learning. Based on Supervised Machine Learning, SVMs can be classified into 2 types; SVC (Support Vector Classification) and SVR (Support Vector Regression). Some applications of SVMs are Face detection, Text and hypertext categorization, Classification of images, Bioinformatics, Protein fold, and remote homology detection, Handwriting detection etc.

1.1 HISTORY

SVM dates back to mid-20th century. "Theory of Reproducing Kernels", published in 1950 by N. Aronszajn, was one of the earliest publications related to SVMs. In the article, Aronszajn wrote about several theories where reproducing a kernel is necessary and gave examples with proof to justify them. In 1957, American psychologist Frank Rosenblatt invented the perceptron, a simple classifier after adopting Aronszajn idea.

Six years later, Vladimir N. Vapnik and Alexey Ya. Chervonenkis invented SVM algorithm (Generalized Potrait). The basis of this class of procedures is an algorithm called Support Vector. This was the true inspiration that led to the 1992 paper by Bernhard Boser, Isabelle Guyon, and Vladimir Vapnik at the COLT conference suggesting a way to create non-linear classifiers by applying the kernel trick to maximum-margin hyperplanes.

2.0 SUPPORT VECTOR MACHINES (SVMs)

The two major applications of SVM in Machine Learning are Classification and Regression (Supervised ML).

2.1 CLASSIFICATION

2.1.1 Linear SVC

To understand the fundamental idea behind linear SVM, the data generated using Scikit-Learn ML Library gives a better explanation (Figure 2.1). For the two-dimensional data, I generated 3 linear boundaries to further explain how the linear SVM models are generated.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=40, centers=2, random_state=0, cluster_std=0.70)
plt.scatter(X[:, 0], X[:, 1], c=y, s=20, cmap='autumn')
plt.show()
```

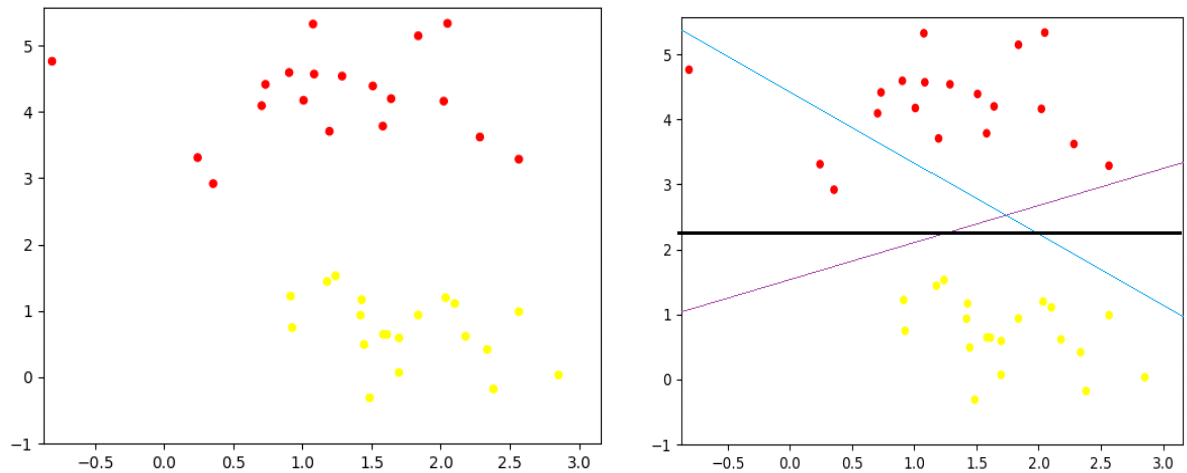

Figure 2.1. Left (a) - dataset containing points with 2 different classes. Right (b) - dataset with boundaries of 3 linear classifiers.

On the right in Figure 2.1 the model whose boundary is represented by a blue line is bad in the fact that it misclassified some points. While the boundary represented by a purple line works perfectly on the training set but being so close to the vectors shows that the model may perform poorly on the validation and test sets. The boundary represented by the solid black line is a true replica of the SVM classifier; the line not only separates the two data perfectly but also maintains good distances from the different training instances. In SVM, the boundary needs to be far away from the closest points as possible. If the number of samples is increased and most points are located far away from the clusters of the points, the boundary isn't affected. This is a special feature of SVM.

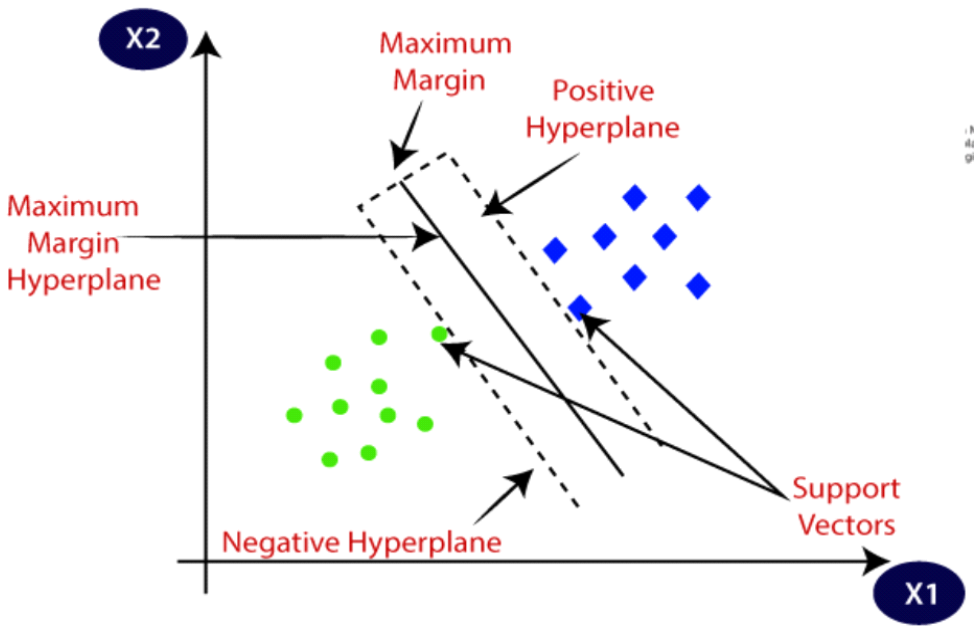

Figure 2.2. SVM features

I know you are all like, "Wow, I love linear SVMs they are so easy to implement". But I will say to you that the above data is likely impossible to get in real life. Most times, you get unstructured data with outliers or data with missing values. Figure 2.2 shows a complicated dataset that will be used to elaborate Linear SVM better.

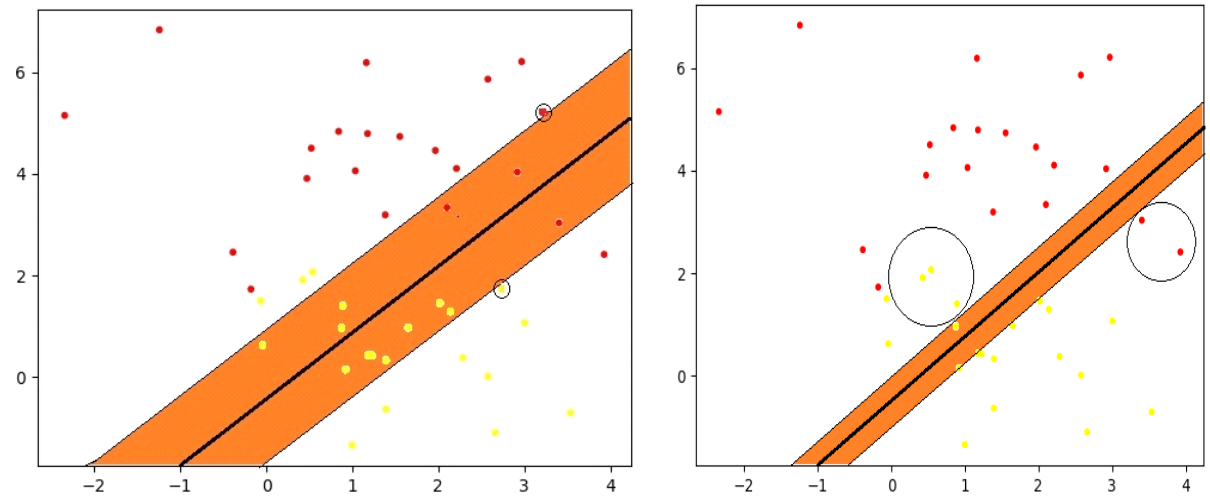

Figure 2.3. Here, there are points both models failed to classify correctly. The model on left is a large margin classifier based on the width of the margin. The points circled are the support vectors. While the other on the right is a small margin classifier.

The Error function of SVMs is calculated below;

Error function = Classification error + Margin error

Classification error occurs when the misclassified points appear on the other side of the boundary but not within the margin, while Margin error occurs when the misclassified points appear on the other side of the boundary but within the margin. In Figure 2.3b (right), the error function will be a result of more classification error (points in the circle) compared to the margin error. While in Figure 2.3a (left), the error function will be a result of more margin error (points within the margin) compared to the classification error. The error function in SVM can be reduced using a process called Gradient Descent.

The classification error can be updated by adding a constant "C parameter".

Error function = C * Classification error + Margin error

The C parameter is a "penalty parameter" of the error term. If C is very large then the error function will occur as a result of classification error and the margin will be very small, while if C is very low then the error function will occur as a result of margin error and the margin will be large. The C parameter is a hyperparameter therefore you have to use GridSearch to get a good value.

Implementation of Linear SVM in Scikit-Kearn is done below.

```
from sklearn.svm import LinearSVC
```

2.1.2 Non-linear SVC

There are situations where linear SVM doesn't perform well, this is when the data is not close to being linearly separable. This is when the robust nature of SVM becomes applicable. The use of kernel hyperparameter. The most used nonlinear kernels are the Radial Basis Function (RBF) and Polynomial Kernel. Polynomial features are easy to implement and it does not only work for SVM, it can be applied to other Machine Learning models. At a low polynomial degree dealing with the complex dataset is likely not possible and at a high polynomial degree, it creates huge datasets making training slow. In SVM, polynomial features can be implemented using the "kernel trick" (Figure 2.4). In SVM the kernel trick makes it possible to implement polynomial features at various degrees without actually adding the features to the training dataset. The training of SVM won't be slow, unlike the dataset where polynomial features are high.

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
poly_kernel_svm_clf = Pipeline([
("scaler", StandardScaler()),
("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)
```

Note: StandardScaler is used to scale the dataset because unlike some ML models SVM is also affected by scaling.

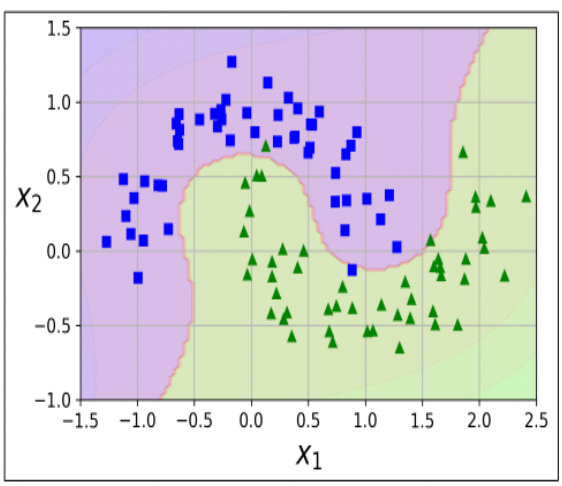

Figure 2.4. Dataset with points separated by polynomial kernel boundary.

The rbf kernel can also be applied to the dataset above and may give a similar result by just changing the kernel to rbf and adding other necessary hyperparameters. There are also more advanced cases of rbf kernel application.

```
rbf_kernel_svm_clf = Pipeline([
("scaler", StandardScaler()),
("svm_clf", SVC(kernel="rbf",gamma=3, C=5))
])
rbf_kernel_svm_clf.fit(X, y)
```

2.2 REGRESSION

2.2.1 Linear and Nonlinear SVR

Just like the SVC, SVR also works the same way but in the reversed case. Instead of the points diverging from the boundary and margin to get a good model, they converge to the boundary and margin to get a good model. This means that in SVR we want as many points within the margin. (Figure 2.5)

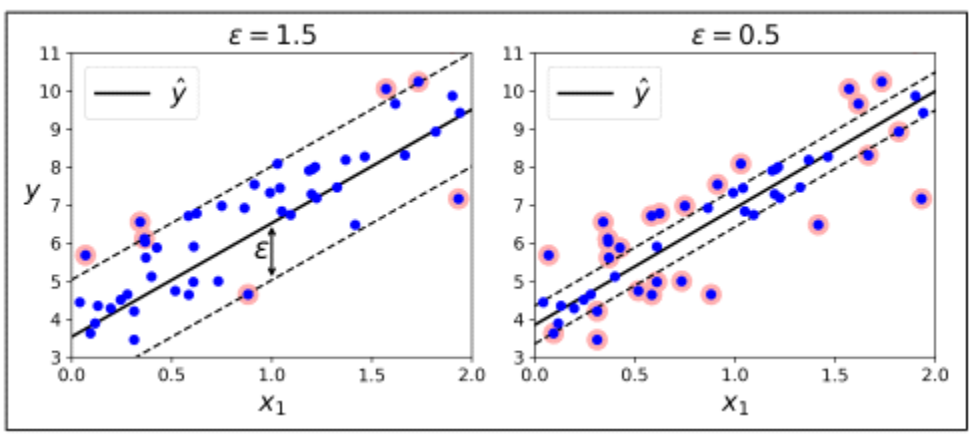

Figure 2.5. Dataset showing boundaries and margin of Linear SVR models with different epsilons

The size of the margin in regression depends on a hyperparameter called epsilon. The greater the size of epsilon the wider the margin and the smaller the size the narrower the margin. Linear SVR can also be implemented in Scikit-Learn.

```
from sklearn.svm import LinearSVR
svr = LinearSVR(eplison = 1.5)
```

For nonlinear regression, the kernel trick is used by setting the kernel hyperparameter to 'poly'.

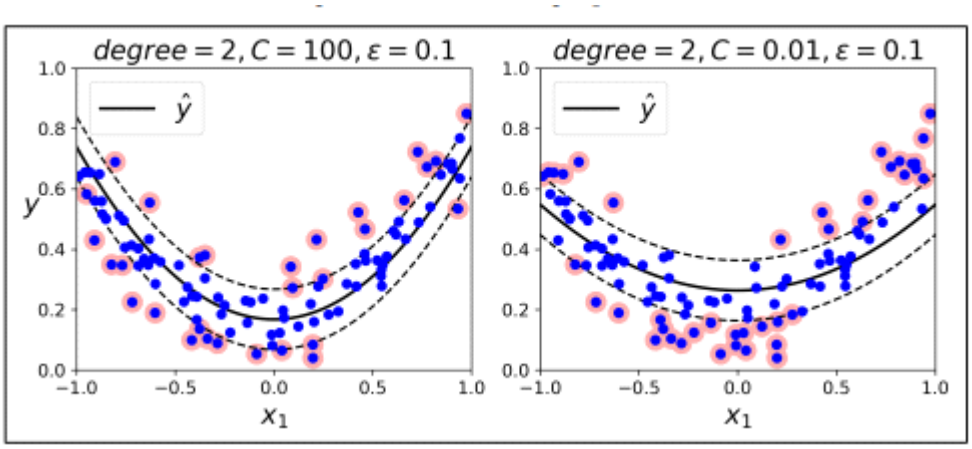

Figure 2.6. Datasets showing boundaries and margins of poly kernel SVR models with different C parameter.

The following code produces the image on the left of Figure 2.6.

```
from sklearn.svm import SVR
svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
```

3.0 SVM VS LOGISTIC REGRESSION

Both SVM and logistic regression are used for binary classification ML problems, but knowing when to use one over the other is the key.

1. SVMs have high computational complexity, therefore it is bad to use them for very large datasets because it will take time to train. Therefore, they are used for relatively small datasets with a lot of features.

2. Logistic regression tends to perform poorly when used on nonlinear datasets, unlike SVMs.

3. They can be applied in regression.

4.0 CONCLUSION

In this article, you have seen how SVM can be implemented in solving several ML problems and although its limitations, it sometimes proves to be a better model for some ML problems. It is also easy to tune to get better performance.

REFERENCES

Chen, L., & Chen, L. (2019). Support Vector Machine-Simply Explained. Towards Data Science.

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems. " O'Reilly Media, Inc.". pp 155 - 175.

HolyPython. (2020, July 29th). Support Vector Machine History. https://holypython.com. https://holypython.com/svm/support-vector-machine-history-/

VanderPlas, J. (2016). Python data science handbook: Essential tools for working with data. " O'Reilly Media, Inc.". pp 405 - 416.

Wikipedia. Edited 20th June 2022 by Billybob195. Support Vector Machine. https://en.m.wikipedia.org. https://en.m.wikipedia.org/wiki/Support-vector_machine